

Support Ops Copilot

An AI-powered support assistant, built in five stages. Every stage adds a capability on top of what's already working — by the end it's one real app, not five disconnected exercises.

FORMAT

One project, one repo

DELIVERABLE

GitHub repo + demo

FRONTEND

Streamlit or custom UI

STAGE 1

FOUNDATIONS

Core AI Service Layer

Build an AIService module with three functions, using role-based and few-shot prompting.

- **Classify** an incoming ticket — category + urgency
- **Draft** a reply to the customer
- **Extract** structured data from a raw message — name, issue, order ID if present

Break it on purpose. Try ambiguous tickets, prompt injection, hallucination bait. Document every failure in `BREAKING_IT.md` — this file matters as much as the code.

All three functions run in the terminal on sample tickets, with honest failure notes.

STAGE 2

RELIABILITY

Make It Production-Safe

Real apps can't trust free-text AI output. Refactor Stage 1 so it's dependable.

- Every function returns **validated JSON** (Zod or Pydantic — your choice)
- Invalid output triggers a **retry** — max 3 attempts, then fail gracefully

- Every response includes a **confidence score**
- **Low-confidence results** get flagged for human review instead of auto-sending

Feed it a deliberately weird ticket — it should not crash, and it should flag itself instead of guessing.

STAGE 3

KNOWLEDGE

Give It Knowledge — RAG

Your assistant doesn't know your product yet.

- Assemble a small knowledge base — FAQ, policy doc, past resolved tickets (synthetic is fine)
- Chunk the docs, generate embeddings, store in **ChromaDB** (local)
- On a ticket, retrieve top-k relevant chunks and generate a grounded answer
- Every AI answer must **cite which chunk/doc** it came from

Ask it something only answerable from your docs — it should answer correctly and show its source.

STAGE 4

AGENTS

Give It Hands — Agent + Tools

Now let it act, not just answer.

- Build a tool-calling agent with at least 3 mock tools — e.g. `lookup_order_status`, `issue_refund`, `escalate_to_human`
- Add a **human-in-the-loop approval gate** before any destructive action
- **Log** every tool call — input and output
- Handle tool errors gracefully — no silent failures

Give it a ticket that needs a refund — it should call the tool, pause for approval, and log the exchange.

STAGE 5

SHIP IT

Bring It Together

One coherent app that routes a ticket end-to-end: **classify** → **retrieve knowledge if needed** → **draft reply or take action** → **flag for review if confidence is low**.

This stage needs a real frontend, not just a terminal. Pick one:

Streamlit

Fastest path to a usable UI — good if you'd rather spend time on the AI logic than on styling.

Custom frontend

HTML/CSS/JS or a framework like React — more work, but a chance to make it look and feel like a product.

- Reviewer can **submit a raw ticket** and see it get classified
- See the **drafted reply**, with source citations if RAG was used
- See **low-confidence tickets flagged**, and approve/reject agent actions from the UI
- See the **tool-call log** somewhere visible
- Draw a **one-page architecture diagram** of how the pieces connect
- Record/present a **5-minute walkthrough**: what you built, key decisions, what you'd change
- **Push the final repo**

Submission Checklist

- ☐ GitHub repo shared, commit history shows progressive build-up
- ☐ README with setup instructions
- ☐ `BREAKING_IT.md` with documented failure cases from Stage 1
- ☐ Validated JSON outputs with retry logic and confidence flagging (Stage 2)
- ☐ Working RAG pipeline with source citations (Stage 3)
- ☐ Tool-calling agent with approval gate and logging (Stage 4)
- ☐ Frontend (Streamlit or custom) with classification, reply, review/approval, and tool-call log visible

☐ Architecture diagram

☐ 5-minute walkthrough delivered

Support Ops Copilot · Capstone Project · One repo, five stages